

Progetto OS161 B: Implementazione Virtual Memory

Gruppo 42

- Greco Eleonora (s354761)
- Jafrancesco Chiara (s355023)
- Lemerle Stefano (s353334)



Esecuzione programmi utente e gestione processi

Supporto al passaggio di argomenti (argc, argv) da command line per esecuzione programmi utente (runprogram.c).

Estensioni per la gestione dei processi:

- Aggiunti campi a struct proc: PID, exit code, semaforo e puntatore al padre.
- Process Table globale protetta da spinlock.

Implementazione syscall:

- sys_getpid
- sys_fork
- sys_exit
- sys_waitpid
- sys_sbrk

Coremap

- Array di struct `coremap_entry` indicizzato per frame.
- Stati tracciati: `FREE`, `IN_USE`, `FIXED`.
- Addrspace proprietario del frame e `vaddr` corrispondente (solo user).
- Canale di attesa `eviction_wchan` per prevenire race condition.

Svolge le seguenti funzioni:

- Inizializza la `coremap` ed `eviction_wchan`, settando i primi frame a `FIXED`.
- Alloca e libera frame su richiesta (più di uno solo per il kernel)
 - Associa un frame all'addrspace proprietario con il suo `vaddr`.
 - Gestisce la logica di rimpiazzamento delle pagine:
 - Algoritmo FIFO, ogni frame utente ha un contatore.
 - Quando si vuole allocare un frame ma la RAM è piena, sceglie il frame con il contatore minore, lo scrive sul disco di swap e poi lo libera (swap-out).

Interfaccia col disco di swap

Utilizzato il disco lhd1:

- Gestione tramite bitmap con mutua esclusione (swap_lock).

Funzioni di basso livello dedicate

- swap_bootstrap: Connette il disco chiamando `vfs_swapon`, ne ottiene la dimensione e inizializza la bitmap.
 - swap_alloc: trova e riserva uno slot libero nella bitmap impostandolo a 1 sotto la protezione di `swap_lock`.
 - swap_free: rilascia lo slot impostandolo a 0.
 - swap_write: effettua la scrittura di una pagina dalla RAM al disco.
 - swap_read: effettua la lettura di una pagina dalla disco al RAM.

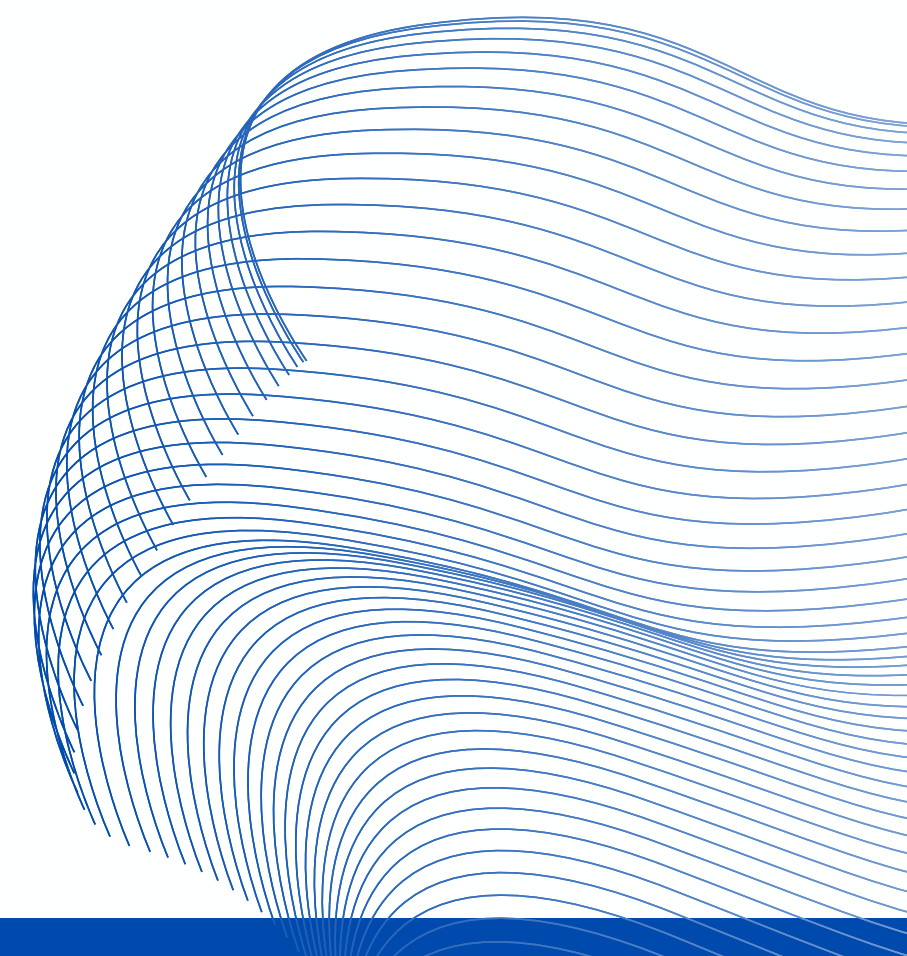
Page Table

Per process page table a due livelli:

- Livello 1 (Page Directory): 1024 puntatori a tabelle di secondo livello (allocate on-demand).
- Livello 2 (Page Table): 1024 entry di tipo `paddr_t`.

Un entry può essere:

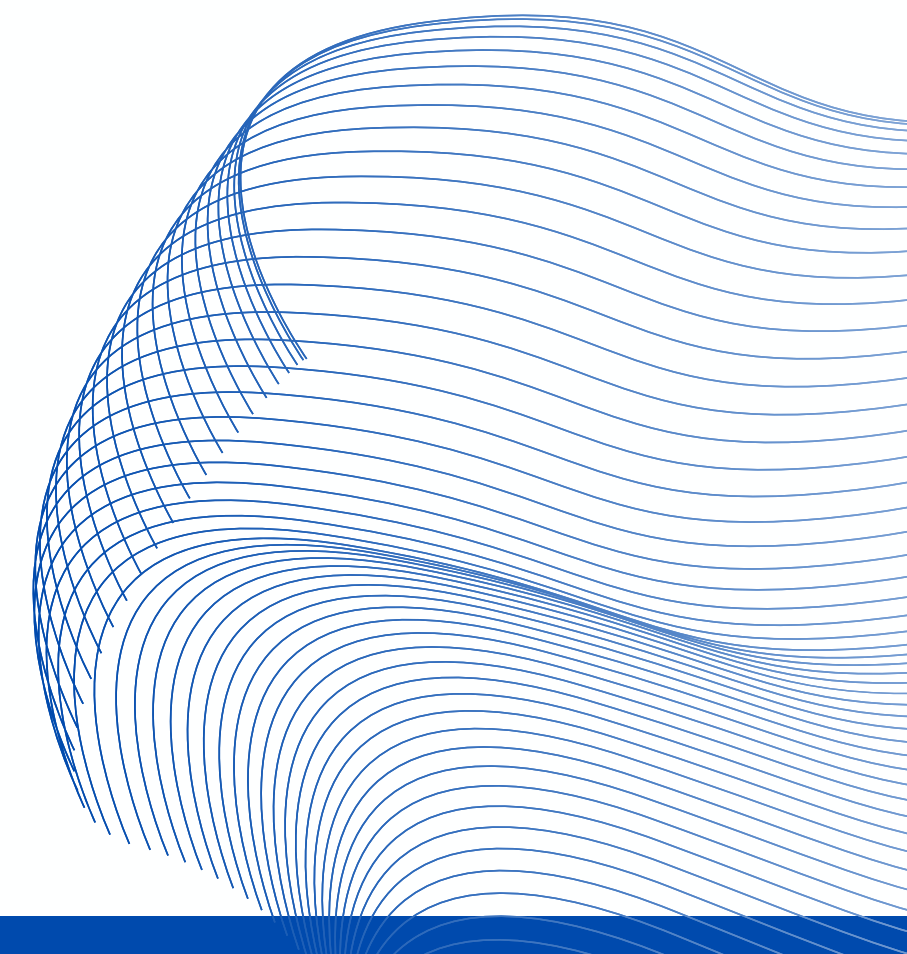
- Nulla: assume il valore di 0.
- Nella RAM: indirizzo del frame fisico nei bit più alti e la flag `PTE_PRESENT = 0x1` nei bit più bassi.
- In swap: nei 12 bit più alti il numero dello slot nel disco di swap nel quale è contenuta la pagina e la flag `PTE_SWAPPED = 0x2` nei bit più bassi.



Page Table

Funzioni chiave:

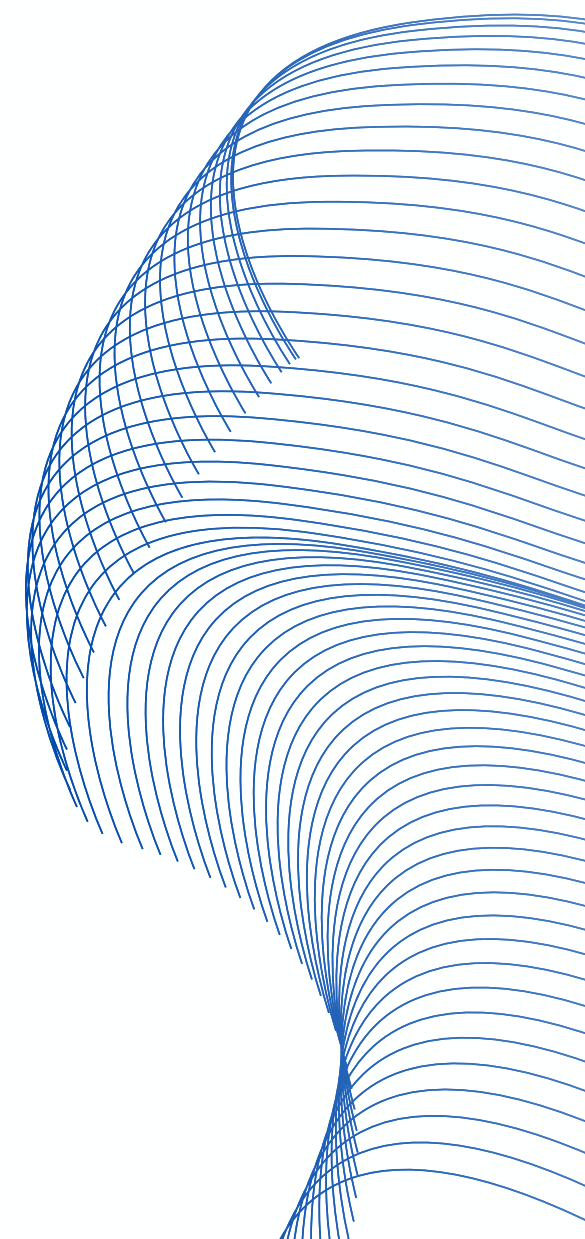
- `pagetable_create/pagetable_create_lv2`: crea una page table di primo/secondo livello.
- `pagetable_destroy`: si occupa di dellocare la page table. Offre sincronizzazione tramite `mem_lock` e distruzione sicura attendendo il risveglio da `eviction_wchan`.
- `pagetable_copy`: deep copy della pagetable dall'address space padre all'address space figlio.
- `pagetable_get_entry`: restituisce un puntatore a `paddr_t`.



Traduzione da vaddr a paddr

pagetable_translate: effettua la Page Table walk traducendo indirizzi logici utente in fisici e risolvendo richieste on-demand.

- Da un vaddr_t estrae gli indici per page directory (lv1) e page table (lv2), se la tabella a lv2 non esiste la alloca.
- Preleva l'entry corrispondente usando gli indici lv1 e lv2, in base all'entry, 3 casi:
 - In RAM: restituisce direttamente l'indirizzo fisico.
 - Primo accesso: alloca un frame, azzerà con bzero, mappa con PTE_PRESENT, registra nella coremap, restituisce l'indirizzo.
 - Su disco: alloca pagina, legge dal disco con swap_read, libera lo slot con swap_free, aggiorna entry con PTE_PRESENT, chiama coremap_set_owner, restituisce il nuovo indirizzo fisico.

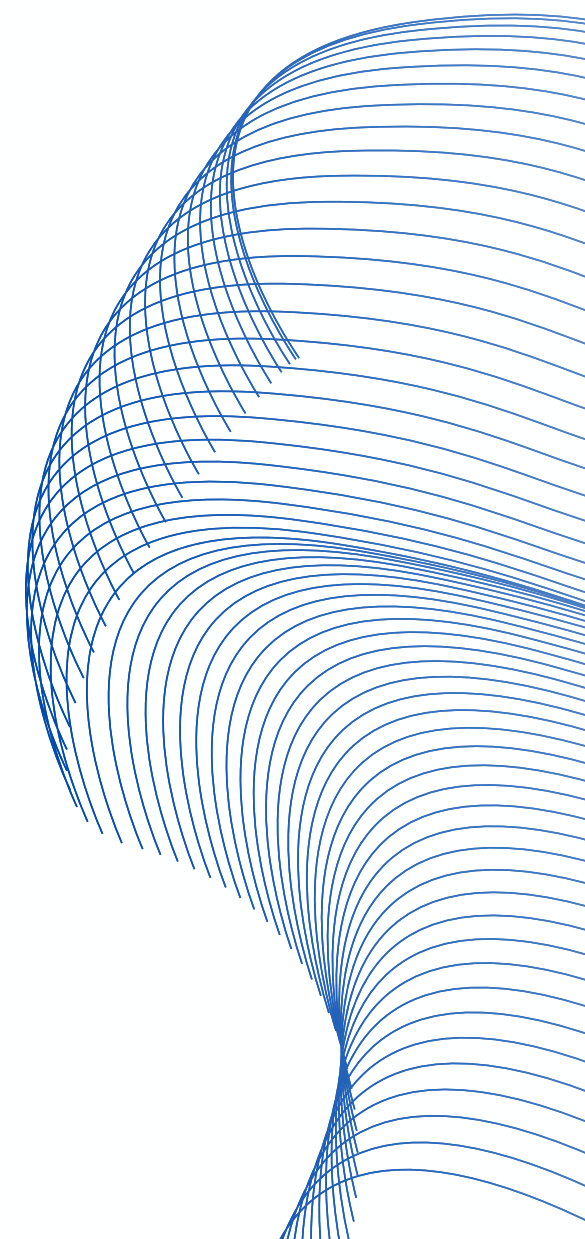


Address space

Ogni processo ha un address space (struct addrspace) contenente una linked list di regioni (con dimensione, indirizzo d'inizio e permessi), informazioni sullo stack e heap, pagetable e flag per dire che sta venendo copiato.

Funzioni:

- `as_create/as_destroy`: alloca/dealloca address space, regioni e page table.
- `as_copy (fork)`: setta `is_copying=true`, duplica pagetable e regioni.
- `as_activate`: flush TLB solo se address space diverso da `active_as`.
- `as_define_region`: aggiunge region alla lista, aggiorna `heap_start`.
- `as_prepare_load/as_complete_load`: writeable temporaneo per `load_elf`, poi ripristina.
- `as_define_stack`: configura stack utente.



Gestore della VM

In vm.c vengono unite tutte le componenti appena definite. Per allocare/deallocare pagine il kernel chiamerà `alloc_kpages/free_kpages` che a sua volta accedono alla `coremap`. All'inizio si chiama `vm_bootstrap` che inizializza la VM.

Ad ogni TLB miss (e per altre eccezioni in memoria), il kernel chiama `vm_fault`, con parametri `faultaddress` e `faulttype`:

- Preleva l'address space, ritorna `EFAULT` se `NULL`.
- Verifica `faultaddress != 0` e `faulttype` valido.
- Scorre le regioni per verificare indirizzo contenuto e permessi compatibili.
- Se non in nessuna regione, controlla heap e stack. Se non trovato: `EFAULT`.
- Chiama `pagetable_translate` per ottenere il paddr.
- Disabilita interrupt, costruisce entry TLB.
- Con `tlb_probe` cerca voce esistente, altrimenti `tlb_write` o `tlb_random`.
- Riabilita interrupt, ritorna 0.

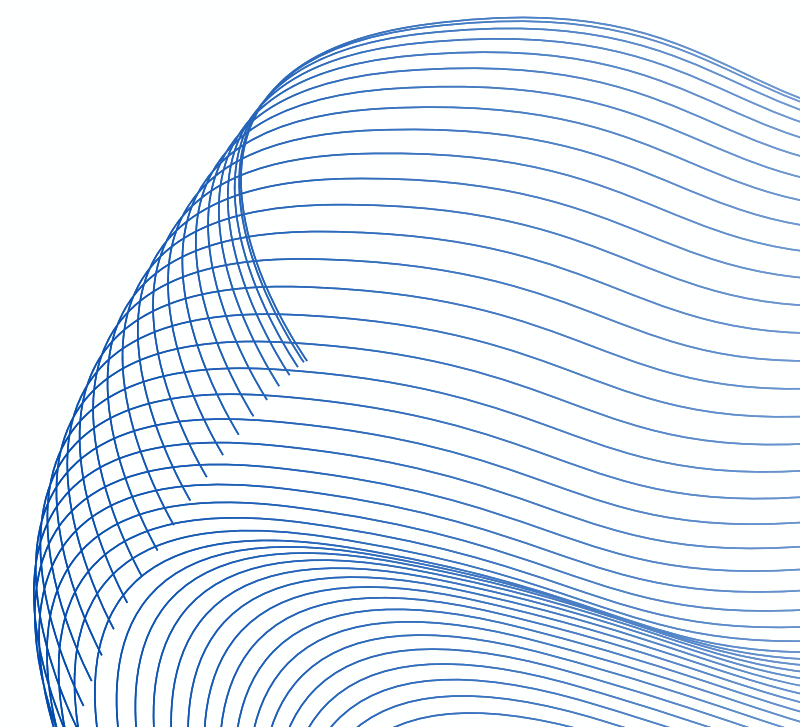


Raccoglimento statistiche

In `vmstats.c` abbiamo introdotto un meccanismo di conteggio statistiche con 6 contatori globali: TLB fault (free/replace), invalidazioni TLB, page fault, page in e page out. Definite funzioni per start/stop/reset/print delle statistiche e `vm_record_stat` per incrementare il contatore appropriato.

Per fornire un'interfaccia a questo meccanismo, abbiamo introdotto a `menu.c` il comando `vmstats`, il quale chiama le funzioni definite nel file `vmstats.c`. Il comando viene invocato scrivendo `vmstats <flag>`:

- `--start`: inizia la registrazione delle statistiche sulla VM.
- `--stop`: termina la registrazione delle statistiche.
- `--reset`: resetta tutti i contatori delle statistiche.
- `--print`: stampa le statistiche raccolte.





Statistiche

Test	RAM size	Tempo (s)	TLB fault (free)	TLB fault (replace)	TLB Invalidation	Page fault	Swap read (page in)	Swap write (page out)
ctest	512K	12248.748	92867	32645	92804	124986	124726	124912
ctest	1M	11602.368	9658	116309	9595	123935	123675	123735
ctest	2M	13.022	64	120321	1	260	0	0
forktest	512K	3.795	221	0	220	28	34	34
forktest	1M	0.844	212	0	72	5	0	0
forktest	2M	0.848	214	0	74	5	0	0
huge	512K	292.087	2896	706	2833	3582	3067	3511
huge	1M	270.589	359	3238	296	3423	2908	3225
huge	2M	231.972	170	3398	107	3155	2640	2702
matmult	512K	58.763	631	193	568	813	430	740
matmult	1M	46.859	106	695	43	772	389	572
matmult	2M	0.686	64	732	1	383	0	0



Statistiche

Test	RAM size	Tempo (s)	TLB fault (free)	TLB fault (replace)	TLB Invalidation	Page fault	Swap read (page in)	Swap write (page out)
malloctest	512K	0.625	7	0	1	7	0	0
malloctest	1M	0.655	7	0	1	7	0	0
malloctest	2M	0.627	7	0	1	7	0	0
palin	512K	15.321	5	0	1	5	0	0
palin	1M	15.321	5	0	1	5	0	0
palin	2M	15.295	5	0	1	5	0	0
parallelVM*	512K	523.375	16858	0	32098	5242	5070	5211
parallelVM	1M	349.415	18806	0	21821	3494	3156	3325
parallelVM	2M	12.830	8615	0	3095	346	5	16
sort	512K	160.582	1354	561	1291	1756	1463	1683
sort	1M	57.310	206	1728	143	814	521	614
sort	2M	3.538	64	1845	1	293	0	0

*Non riesce ad effettuare tutte e 24 le fork, spiegato meglio dopo



Analisi dei risultati

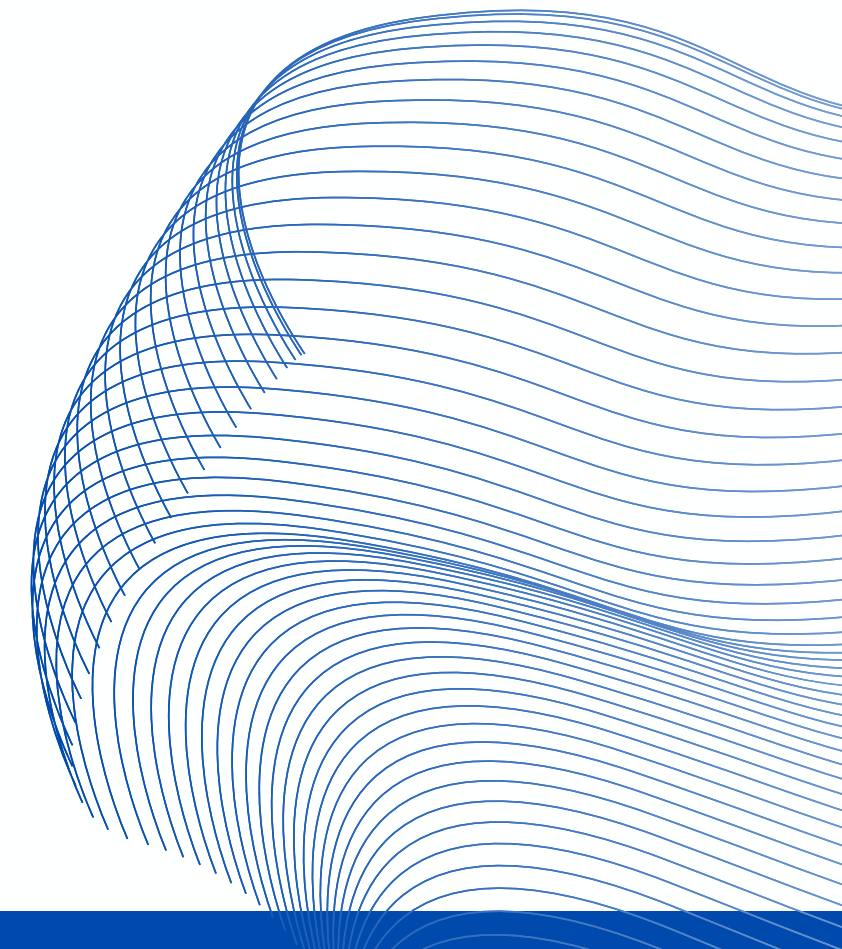
Il fenomeno del Thrashing (ctest & huge):

- L'accesso a balzi distrugge il principio di località spaziale.
- Sotto i 2MB di RAM, il rimpiazzamento FIFO porta a un continuo I/O su disco (tempo di esecuzione di ctest > 3 ore).

I limiti della memoria kernel (parallelVM):

- Con 512K di RAM, 11 processi su 24 falliscono per Out Of Memory. Le pagine adibite allo stack del kernel sono FIXED e non swappabili.

I test passano tutti e le statistiche mostrano un funzionamento corretto e stabile della VM anche in scenari concorrenti e con molto stress sul sistema.





Grazie per l'attenzione